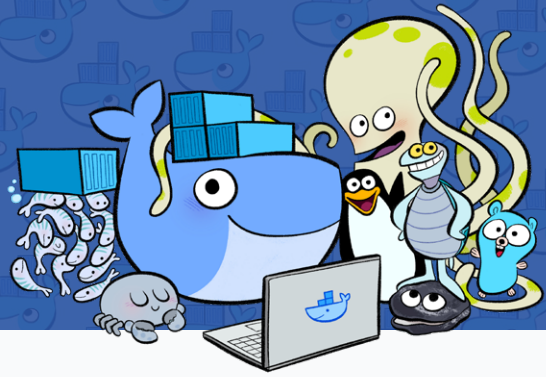




CLI Cheat Sheet



Docker provides the ability to package and run an application in a loosely isolated environment called a container. The isolation and security allows you to run many containers simultaneously on a given host. Containers are lightweight and contain everything needed to run the application, so you do not need to rely on what is currently installed on the host. You can easily share containers while you work, and be sure that everyone you share with gets the same container that works in the same way.

INSTALLATION

Docker Desktop is available for Mac, Linux and Windows
<https://docs.docker.com/desktop>

View example projects that use Docker
<https://github.com/docker/awesome-compose>

Check out our docs for information on using Docker
<https://docs.docker.com>

IMAGES

Docker images are a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings.

Build an Image from a Dockerfile

```
docker build -t <image_name> .
```

Build an Image from a Dockerfile without the cache

```
docker build -t <image_name> . --no-cache
```

List local images

```
docker images
```

Delete an Image

```
docker rmi <image_name>
```

Remove all unused images

```
docker image prune
```

DOCKER HUB

Docker Hub is a service provided by Docker for finding and sharing container images with your team. Learn more and find images at <https://hub.docker.com>

Login into Docker

```
docker login -u <username>
```

Publish an image to Docker Hub

```
docker push <username>/<image_name>
```

Search Hub for an image

```
docker search <image_name>
```

Pull an image from a Docker Hub

```
docker pull <image_name>
```

GENERAL COMMANDS

Start the docker daemon

```
docker -d
```

Get help with Docker. Can also use `--help` on all subcommands

```
docker --help
```

Display system-wide information

```
docker info
```

CONTAINERS

A container is a runtime instance of a docker image. A container will always run the same, regardless of the infrastructure. Containers isolate software from its environment and ensure that it works uniformly despite differences for instance between development and staging.

Create and run a container from an image, with a custom name:

```
docker run --name <container_name> <image_name>
```

Run a container with and publish a container's port(s) to the host.

```
docker run -p <host_port>:<container_port> <image_name>
```

Run a container in the background

```
docker run -d <image_name>
```

Start or stop an existing container:

```
docker start|stop <container_name> (or <container-id>)
```

Remove a stopped container:

```
docker rm <container_name>
```

Open a shell inside a running container:

```
docker exec -it <container_name> sh
```

Fetch and follow the logs of a container:

```
docker logs -f <container_name>
```

To inspect a running container:

```
docker inspect <container_name> (or <container_id>)
```

To list currently running containers:

```
docker ps
```

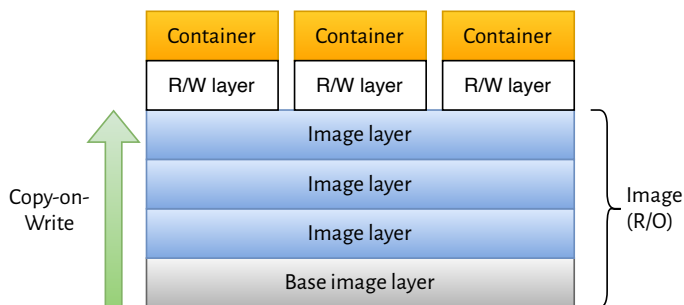
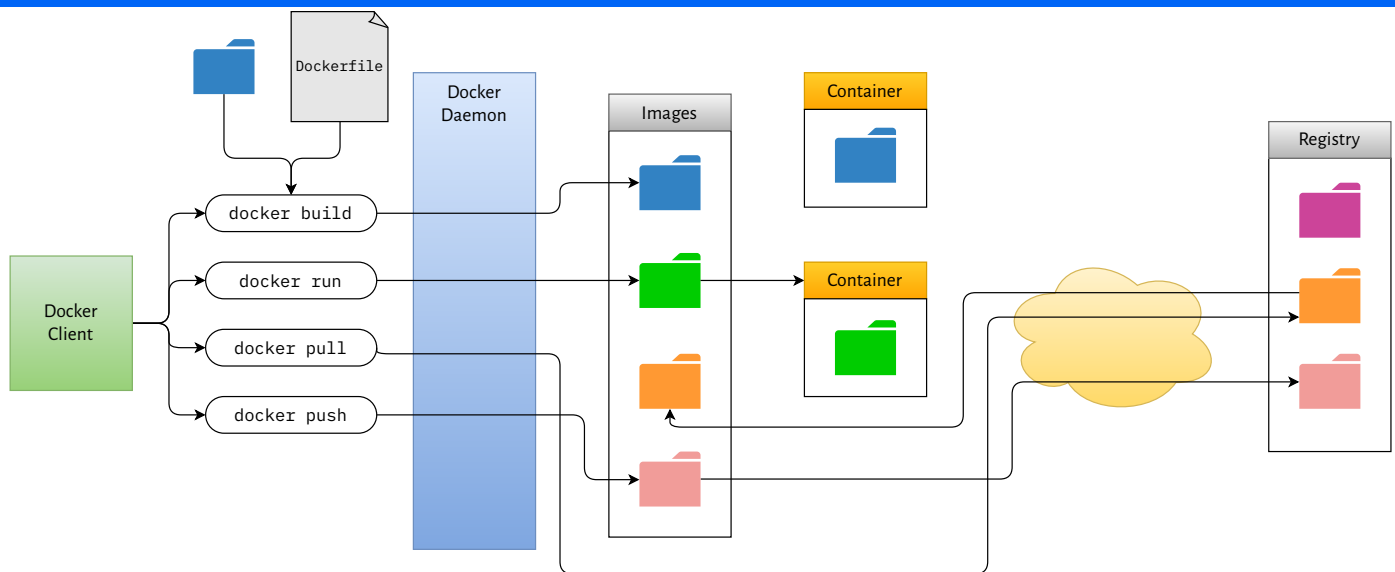
List all docker containers (running and stopped):

```
docker ps --all
```

View resource usage stats

```
docker container stats
```

Architecture



Storage Drivers

Driver	FS	Level
overlay2	ext4, XFS (ftype=1)	File
fuse-overlayfs	Any	File
btrfs	BTRFS	Block
zfs	ZFS on Linux (ZoL)	Block
vfs	Any (no CoW)	File

Images

<code>docker images <repo>:<tag></code>	List all images in <code><repo></code> filtered by <code><tag></code> (both are optional)
<code>... -a</code>	Show all images (i.e. include intermediate images)
<code>... -f "dangling=true"</code>	Leaf images with no tags attached (e.g. if removed by later build)
<code>... -f "label=<value>"</code>	Show images with attached label
<code>... --no-trunc</code>	Don't truncate image IDs
<code>... --digests</code>	Show digests
<code>docker inspect <image></code>	Display detailed information about <code><image></code>
<code>docker rmi <image></code>	Deletes local image <code><image></code>
<code>docker image prune</code>	Deletes all dangling local images (i.e. those not used by a named image)
<code>... -a</code>	Also delete unused local images (i.e. those not referenced by a container)
<code>docker history <image></code>	Show the layers used to build a specified image



Building

<code>docker build -t <image> <dir></code>	Builds <image> using build context <dir> containing Dockerfile
<code>docker build -t <image>:<tag> .</code>	Builds image <image> with tag <tag> using current dir as build context
<code>... --builder <name></code>	Use alternative building instance <name>
<code>... --build-arg <var>=<value></code>	Specify value for parameter declared with ARG in Dockerfile
<code>... -f <path></code>	Look for Dockerfile at <path> instead of <dir>/Dockerfile
<code>... --no-cache</code>	Bypass the build cache
<code>... --cache-from=<src></code>	Use <src> as source for cached images
<code>... --cache-to=<dst></code>	After build, exports intermediate images to cache <dst>
<code>... --rm</code>	Remove any intermediate containers after successful build
<code>... --secret id=<id>,src=<file></code>	Exposes <file> as secret <id> to match a type=secret mount
<code>docker buildx create <opts> <name></code>	Creates a new builder instance
<code>... --name <name> --append</code>	Instead of creating a new builder, append new node to builder <name>
<code>... --driver <name></code>	Use build driver <name>
<code>... --driver-opt "<opt>=<val>,..."</code>	Pass driver-specific options as comma-separated <opt>=<val> list
<code>... --use</code>	Also switch to newly-created building, as with docker buildx use
<code>docker buildx use <name></code>	Use specified builder from now on
<code>... --default</code>	Set building as default for current context
<code>... --global</code>	Builder persists even after context is changed
<code>docker buildx ls</code>	Lists existing builder instances, with current one marked with ★
<code>docker buildx rm <name></code>	Removes the specified (or current if omitted) builder instance

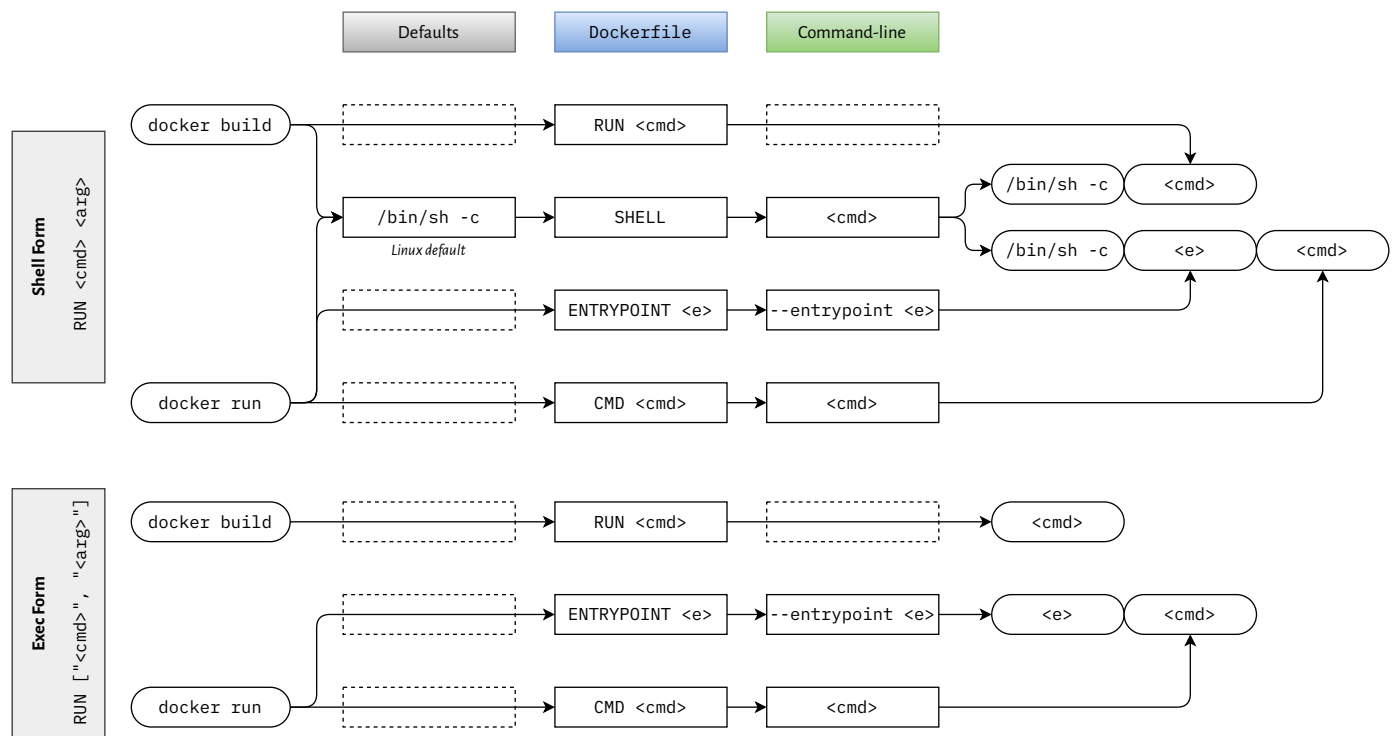
Driver	Notes	Auto load image	Cache export	Tarball output	Multi-arch images	BuiltKit configuration
docker	Uses bundled BuildKit (default)	✓				
docker-container	Creates BuildKit in container		✓	✓	✓	✓
kubernetes	Creates BuildKit in Kubernetes cluster		✓	✓	✓	✓
remote	Connects to remote BuildKit daemon		✓	✓	✓	External

Driver Options: docker-container

image	Sets BuildKit image to use in container	cpuset-cpus	Limits the set of CPU cores used by container
memory	Set maximum memory usage of container	cpuset-mems	Limits set of CPU memory nodes available
memory-swap	Sets swap memory limit for container	network	Set network mode for the container
cpu-quota	Imposes CPU CFS quota on container	cgroup-parent	Only when using cgroupfs driver
cpu-period	Sets CPU CFS scheduler period on container	restart-policy	See restart in compose reference
cpu-shares	Configures CPU share (relative) of container	env.<var>	Sets value of environment <var>



Shell Form vs. Exec Form



Containers

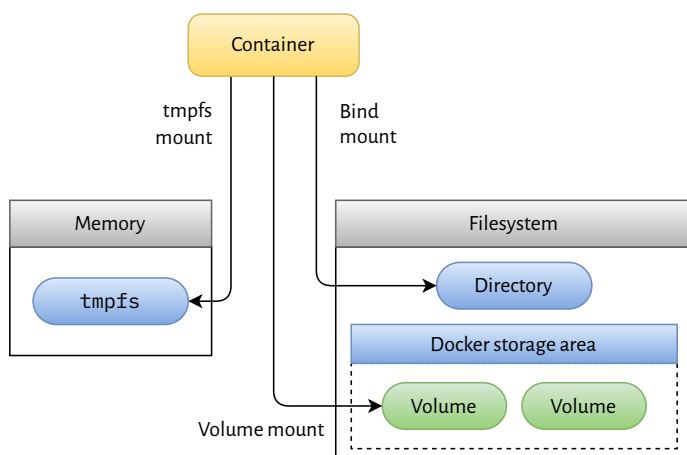
<code>docker run <image></code>	Starts container with <image> and runs CMD from Dockerfile
<code>docker run <image> <cmd></code>	Starts container with <image> and runs <cmd>
<code>... -a <stream></code>	Attach only to <stream> (default is stdout and stderr)
<code>... -d or --detach</code>	Detached mode: run container in background, print container ID
<code>... -e <var>=<value></code>	Set environment variable <var> to <value> (or current value if omitted)
<code>... --entrypoint <cmd></code>	Override the ENTRYPOINT from Dockerfile
<code>... -h <hostname></code>	Set the hostname in the container to <hostname>
<code>... -i -t</code>	Interactive mode (keep stdin open) and allocate pseudo-TTY
<code>... --ip <addr4> --ip6 <addr6></code>	Sets IPv4 address to <addr4> and IPv6 address to <addr6>
<code>... -m <size></code>	Set memory limit to <size> (e.g. 512MB or 2GB)
<code>... --mount type=bind,src=<s>,dst=<d></code>	Bind mount <s> on host into <d> within container
<code>... --mount type=volume,src=<n>,dst=<d></code>	Bind volume named <n> on host into <d> within container
<code>... --name <name></code>	Set container name to <name>
<code>... --network=<net></code>	Connect container to <net>, created with <code>docker network create</code>
<code>... -p <ip>:<hport>:<cport>/tcp</code>	Bind container TCP port <cport> to <hport> of host interface <ip>
<code>... --read-only</code>	Mount container's root filesystem as read-only
<code>... -w <dir></code>	Set current working directory in the container to <dir>



Containers (cont.)

<code>docker attach <container></code>	Attach terminal to standard in/out/error of command in <code><container></code>
<code>docker cp <container>:<src> <dst></code>	Copy file <code><src></code> in <code><container></code> to <code><dst></code> on host
<code>docker commit <container> <image></code>	Write <code><container></code> current filesystem to <code><image></code>
<code>docker create <image></code>	As <code>docker run</code> , but doesn't start the container
<code>docker diff <container></code>	Show changes to filesystem in <code><container></code> since it was created
<code>docker exec <container> <cmd></code>	Run additional command <code><cmd></code> in <code><container></code>
<code>docker inspect <container></code>	Show detailed information about <code><container></code> in JSON
<code>docker kill -s SIGHUP <container></code>	Send signal <code>SIGHUP</code> to command in <code><container></code> (default <code>SIGKILL</code>)
<code>docker logs --follow <container></code>	Show logged <code>stdout</code> in <code>stderr</code> data, and continue to follow them
... <code>--since <datetime></code>	Show all logs captured after <code><datetime></code> (either ISO, or <code>15m</code> , <code>30s</code> , etc.)
... <code>--tail <lines></code>	Show only the most recent <code><lines></code> lines
<code>docker ps -a</code>	Show all containers (or just running ones without <code>-a</code>)
<code>docker ps --last <n></code>	Show only <code><n></code> most recently created containers (also implicitly sets <code>-a</code>)
<code>docker pause <container></code>	Pause execution of all processes in <code><container></code> as if with <code>SIGSTOP</code> .
<code>docker unpause <container></code>	Restarts execution previously stopped with <code>docker pause</code>
<code>docker port <container></code>	Display port mappings for <code><container></code>
<code>docker rename <old> <new></code>	Rename container <code><old></code> to <code><new></code>
<code>docker restart <container></code>	Restart <code><container></code>
<code>docker rm <container></code>	Delete <code><container></code>
<code>docker stop <container></code>	Stop <code><container></code> with <code>SIGTERM</code> , then <code>SIGKILL</code> after timeout
<code>docker start <container></code>	Start stopped <code><container></code>
<code>docker stats</code>	Display a live data string for running containers
<code>docker top <container></code>	Display the running processes in <code><container></code>

Mounts



- Volumes use a **volume driver** to store volumes
- Default volume driver is **local**
- Takes no options on Windows
- On Linux, takes options similar to `mount` command

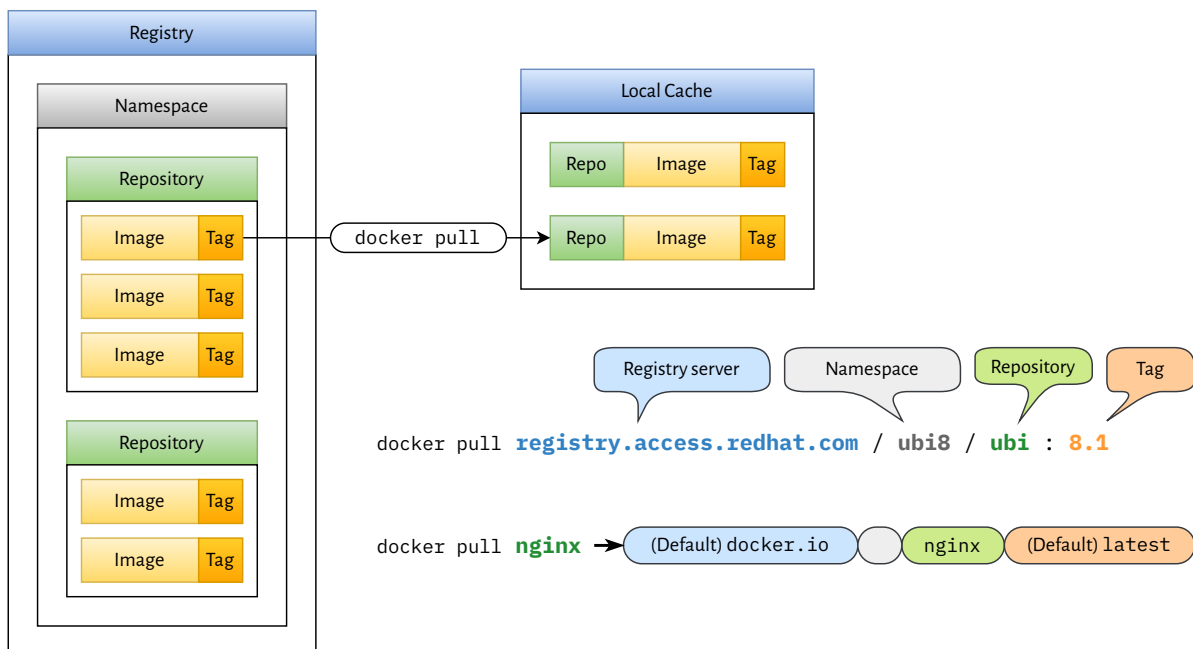

```
docker volume create --driver local \
  --opt type=nfs \
  --opt o=addr=1.2.3.4,rw \
  --opt device=:/remote/path
```
- Further volume drivers are added by volume plugins
- Run `docker info` and look for **Plugins: /Volume:**



Volumes

<code>docker volume create <vol></code>	Create volume <vol> (if omitted, Docker generates a random name)
<code>... -d <driver></code>	Use alternative driver <driver> (typically requires volume plugins)
<code>... -o <option>=<value></code>	Pass options directly to the volume driver (see Mounts section above)
<code>docker volume inspect <vol></code>	Displays information about <vol> as JSON
<code>docker volume ls</code>	List all currently known volumes
<code>docker volume prune -a</code>	Remove volumes not used by any containers (without -a , only unnamed)
<code>docker volume rm <vol></code>	Delete volume <vol> , will fail if used by at least one container

Registries



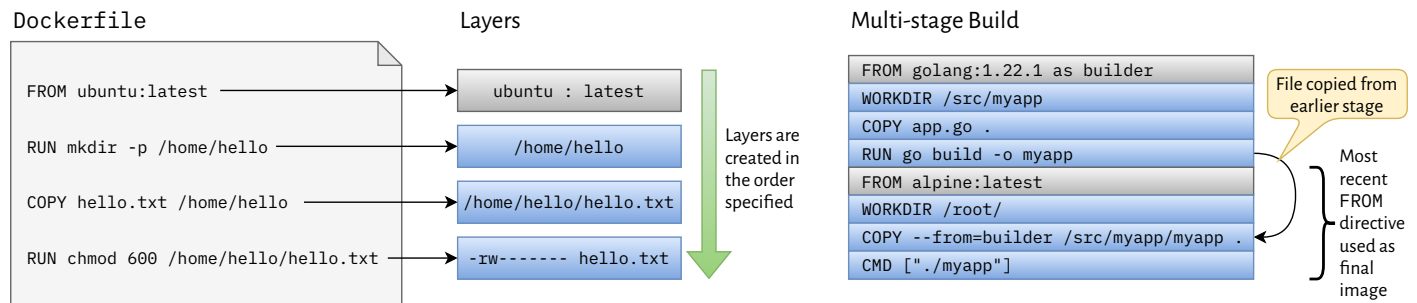
<code>docker pull <name>:<tag></code>	Download image from repository <name> at tag <tag>
<code>... -a</code>	Pull all tags within the repository
<code>docker login <host></code>	Log in to a specified registry
<code>docker tag <i> <host>/<repo>:<tag></code>	Tag image <i> for pushing to <repo> on <host> with tag <tag>
<code>docker push <host>/<repo>:<tag></code>	Upload image <repo>:<tag> to same repository on registry <host>
<code>docker search <host>/<term></code>	Search for <term> on registry <host> (Docker Hub if not specified)

Cache Source/Destination Specifications

<code>type=local,src=<path>,tag=<tag></code>	Push / pull from local directory (s/src/dest/ on export)
<code>type=registry,ref=<host>/<repo>:<tag></code>	Push / pull from remote registry
<code>type=inline</code>	Embed the cache in the image, and push them both together
<code>mode=max</code>	Include intermediate layers (export only)
<code>ignore-error=true</code>	Ignore errors (export only)



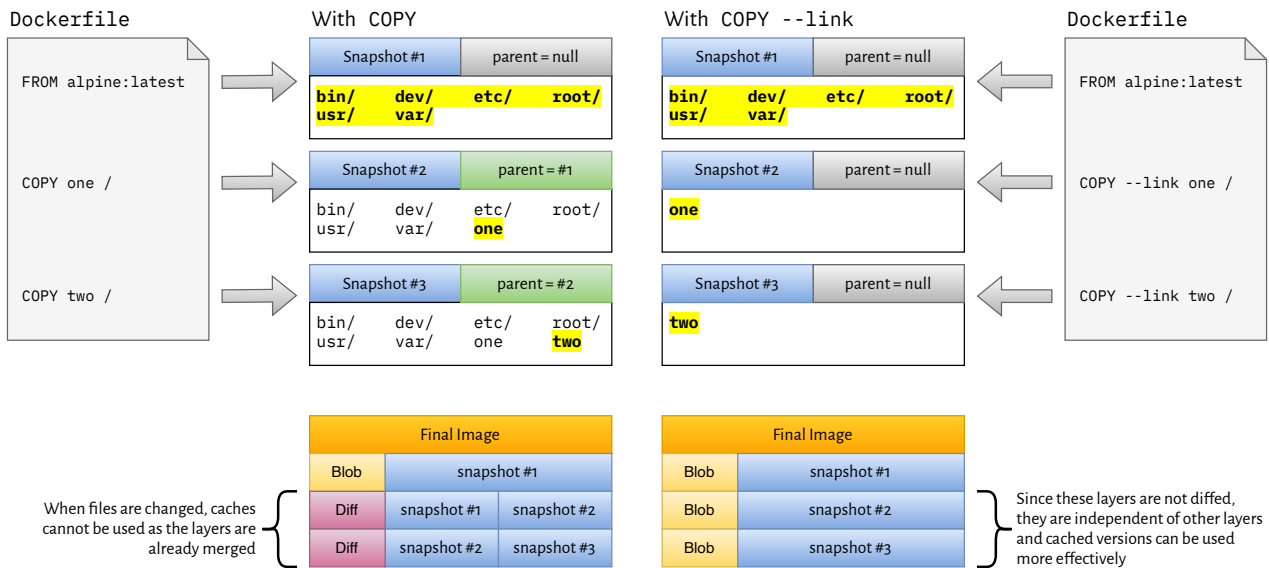
Dockerfile



FROM <image>:<tag> AS <name>	Start new stage named <name> with <image>:<tag> as base image
... --platform=<platform>	Select <platform> from a multi-platform image
RUN ["<cmd>", "<arg>", ...]	Execute <cmd> <arg> ... when building image (exec form)
RUN <cmd> <arg>	Execute <cmd> <arg> ... when building image (shell form)
RUN <<EOF	Execute multiple commands in one step (shell form, heredoc)
#!/usr/bin/env python	Optionally, a shebang line can define an alternative shell
<cmd>	Each command is run with the specified shell
<cmd>	The entire block is considered one "command" and creates one layer
EOF	Terminate with a line containing <i>only</i> delimiter specified on the first line
RUN --mount=type=bind,from=<stage>,source=<src>,target=<dst> <cmd>	Bind mounts <src> in earlier stage <stage> (or build context if omitted) at <dst> in container for running <cmd>
<i>Tip: Bind mounted files only persist for a single instruction and so the file doesn't exist in the final image, which can be more efficient.</i>	
RUN --mount=type=cache,target=<dst>,sharing=locked <cmd>	Mount empty directory for caching at <dst> in container for running <cmd> using single-writer locking (default is shared)
RUN --mount=type=tmpfs,target=<dst>,size=<size> <cmd>	Mount a tmpfs at <dst> limited to <size> in container for running <cmd>
RUN --mount=type=secret,id=<id>,target=<dst> <cmd>	Mount secret <id> as <dst> in container, without including secret in the image (see --secret option to docker build)
RUN --mount=type=ssh <cmd>	Allow container to access keys from host SSH agent for running <cmd>
... --network=<type>	Specify <type> as none for no network, host for host's network
CMD ["<cmd>", "<arg>", ...]	Set default run command to <cmd> (exec form)
CMD <cmd> <arg>	Set default run command to <cmd> (shell form)
ENTRYPOINT <cmd>	Set prefix for run commands (exec and shell forms, see diagram p.2)
SHELL ["<cmd>", ...]	Override platform's default shell (exec form only, see diagram p.2)
COPY <src> ... <dst>	Copies file(s) <src> (relative to context) into <dst> (relative to PWD)
... --from=<src>	Instead of build context, copy from specified stage or other image
... --chown=<u>:<g> --chmod=<p>	Specify owner user <u>, group <g> and permissions <p> of target
... --link	Places copied files into their own snapshot layer for better build caching
WORKDIR <dir>	Sets PWD, if not absolute then relative to existing PWD

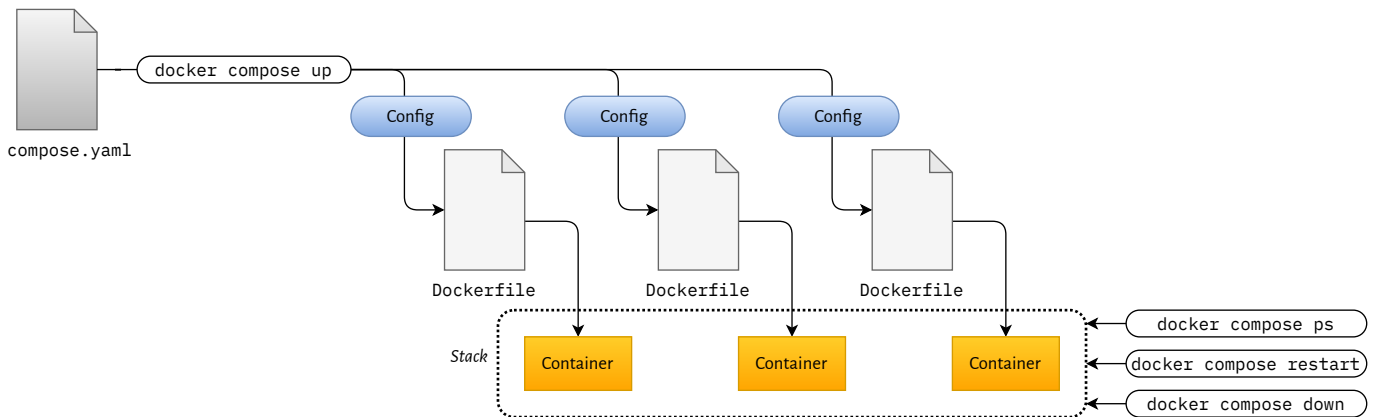


Dockerfile (cont.)



ADD <src> ... <dir>	Similar to COPY but supports fetching from remote URLs as well
ADD http://eg.com/x.txt dst/	Due to trailing slash, creates files dst/x.txt
ADD foo.tar <dir>	Will be unpacked in <dir> (also if compressed with gzip , bzip2 and xz)
... --link --chown --chmod	Same meaning as for COPY
... --checksum=<algo>:<hash>	Validate hash of fetched file with <algo> (e.g. sha256) matches <hash>
ARG <var>=<default>	Declare build arg <var> with default, reference later with \$<var>
docker build --build-arg <var>=<value> ... to override the value later, at build time	
ENV <var>=<value>	Sets environment variable <var> to <value> both at build- and runtime
Tip: If only required at build time, consider using ARG, or just setting for just one command, such as: RUN VAR=value cmd	
EXPOSE <port>/<proto>	Documents <port> as a listen port, <proto> is tcp if omitted
Tip: This doesn't actually publish ports to the host, but passing -P to docker run will publish all exposed ports to random host ports.	
HEALTHCHECK CMD <cmd>	Specifies health check command, or NONE instead of CMD to disable
... --interval=<duration>	Run this time after start, and again at each interval (default 30s)
... --timeout=<duration>	A check taking longer than this is considered failed (default 30s)
... --retries=<n>	Consider container unhealthy after <n> consecutive failures (default 3)
LABEL <key>=<value> ...	Adds metadata to image, use double quotes as needed
MAINTAINER <name>	Sets the Author field, but in general LABEL should be used instead
ONBUILD <instruction>	Adds a trigger to be executed as if just after FROM in a derived build
STOPSIGNAL <signal>	Override default SIGTERM sent by docker stop
USER <user>:<group>	Sets default user and (optionally) group for remainder of stage
VOLUME [<"<dir">"]	Creates specified mount point linked to new anonymous volume on host

Docker Compose



<code>docker compose up</code>	Build, (re)create, start containers, and attach to the merged output of them
<code>... --abort-on-container-exit</code>	Stops all containers once any container exits (cannot be used with <code>-d</code>)
<code>... -d or --detach</code>	Detached mode: run containers in the background
<code>... --force-recreate</code>	Re-create containers even if the image and configuration are the same
<code>... --no-recreate</code>	Don't re-create containers which already exist
<code>... --wait</code>	Used with detached mode, wait for containers to be running and healthy
<code>... -w or --watch</code>	Watch sources and rebuild/refresh containers on changes
<code>docker compose down</code>	Stops containers and removes containers, networks, volumes and images
<code>... --remove-orphans</code>	Remove containers for services no longer defined in Compose file
<code>... --rmi</code>	Remove images used by services
<code>... -v or --volumes</code>	Remove volumes names in Compose file & attached anonymous volumes
<code>docker compose build</code>	Runs only the build step
<code>docker compose config</code>	Display final configuration that will be applied
<code>... --format=json</code>	Renders in JSON instead of YAML
<code>... -o <file></code>	Outputs to <code><file></code> instead of <code>stdout</code>
<code>... --images</code>	Just display list of images, each on one line
<code>... --services</code>	Just display list of service names, each on one line
<code>... --volumes</code>	Just display list of volume names, each on one line
<code>docker compose cp <svc>:<src> <dst></code>	Copy files from <code><src></code> in container for <code><svc></code> to local <code><dst></code>
<code>docker compose create</code>	Runs only the container creation step
<code>docker compose events</code>	Stream events for all containers in the stack
<code>... --json</code>	Render events in JSON format
<code>docker compose exec <svc> <cmd></code>	Equivalent to <code>docker exec</code> in container of specified <code><svc></code>
<code>docker compose images</code>	Lists images used by the containers in the stack
<code>docker compose kill</code>	Force stop containers



Docker Compose (cont.)

<code>docker compose logs</code>	Displays log output from all services
<code>docker compose ls</code>	Lists running Compose projects
<code>docker compose pause</code>	Pauses all containers in the project
<code>docker compose unpause</code>	Unpauses containers paused with <code>docker compose pause</code>
<code>docker compose port <svc> <port></code>	Prints the public port bound to private <code><port></code> in <code><svc></code>
<code>docker compose ps</code>	Lists all containers for the project, including status and exposed ports
<code>docker compose pull</code>	Pulls images associated with services, but doesn't start containers
<code>docker compose push</code>	Pushes locally built images to their respective registries
<code>docker compose restart</code>	Restarts all stopped and running services in the project
<code>docker compose rm</code>	Removes stopped containers from the project
... <code>-s</code> or <code>--stop</code>	Also stops containers, if required, before removing
... <code>-v</code> or <code>--volumes</code>	Remove any anonymous volumes attached to the containers
<code>docker compose run <svc> <cmd></code>	Starts specified <code><svc></code> from the project and runs <code><cmd></code> in it
... <code>-d</code> or <code>--detach</code>	Detached mode: run containers in the background
... <code>--rm</code>	Remove container when it exits
... <code>-P</code> or <code>--service-ports</code>	Also map all service's ports from the compose file to the host
... <code>-w <dir></code>	Set current working directory in the container to <code><dir></code>
<code>docker compose start</code>	Starts existing containers for services in the project
<code>docker compose stop</code>	Stops running containers in the project without removing them
<code>docker compose top</code>	Displays running processes in all running containers in projects
<code>docker compose wait <svc></code>	Blocks until the first of the specified services stops
<code>docker compose watch</code>	Watches build contexts for services and rebuild/restart when they change

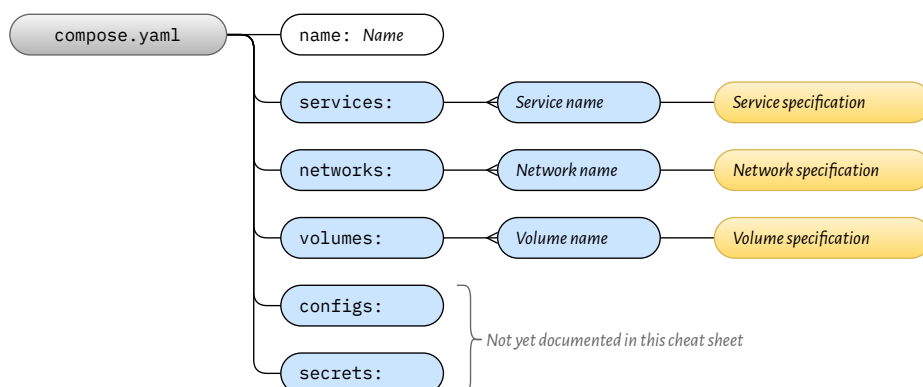
Docker Compose File

Key:

Leaf node

Parent node

Subtree shown elsewhere

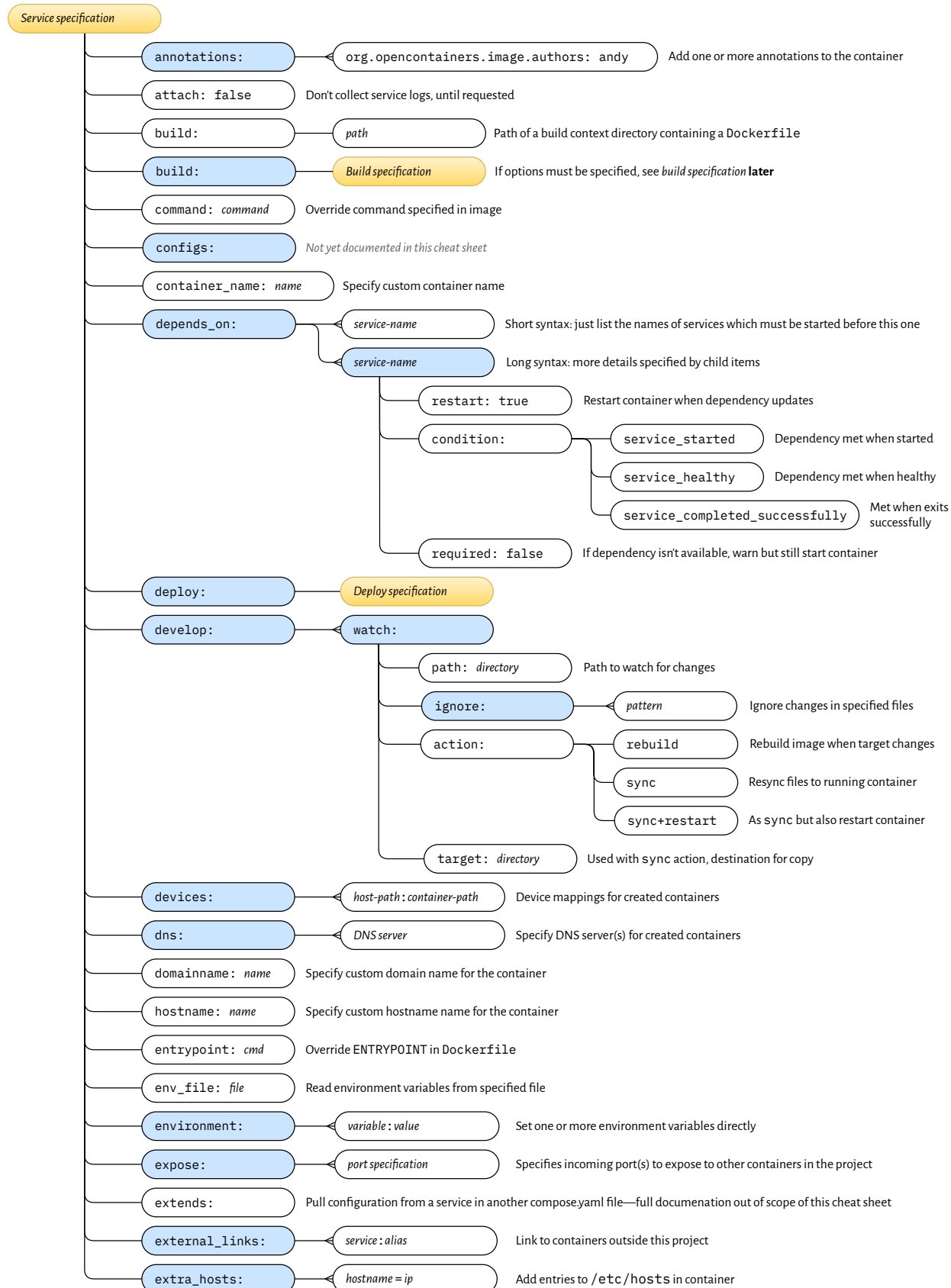


Only some options are shown, this is not a comprehensive reference. Latest version: <https://www.andy-pearce.com/docker-cheat-sheet.pdf>

Docker and the Docker logo are trademarks or registered trademarks of Docker, Inc. in the USA and other countries.



Docker Compose File (Service Specification)

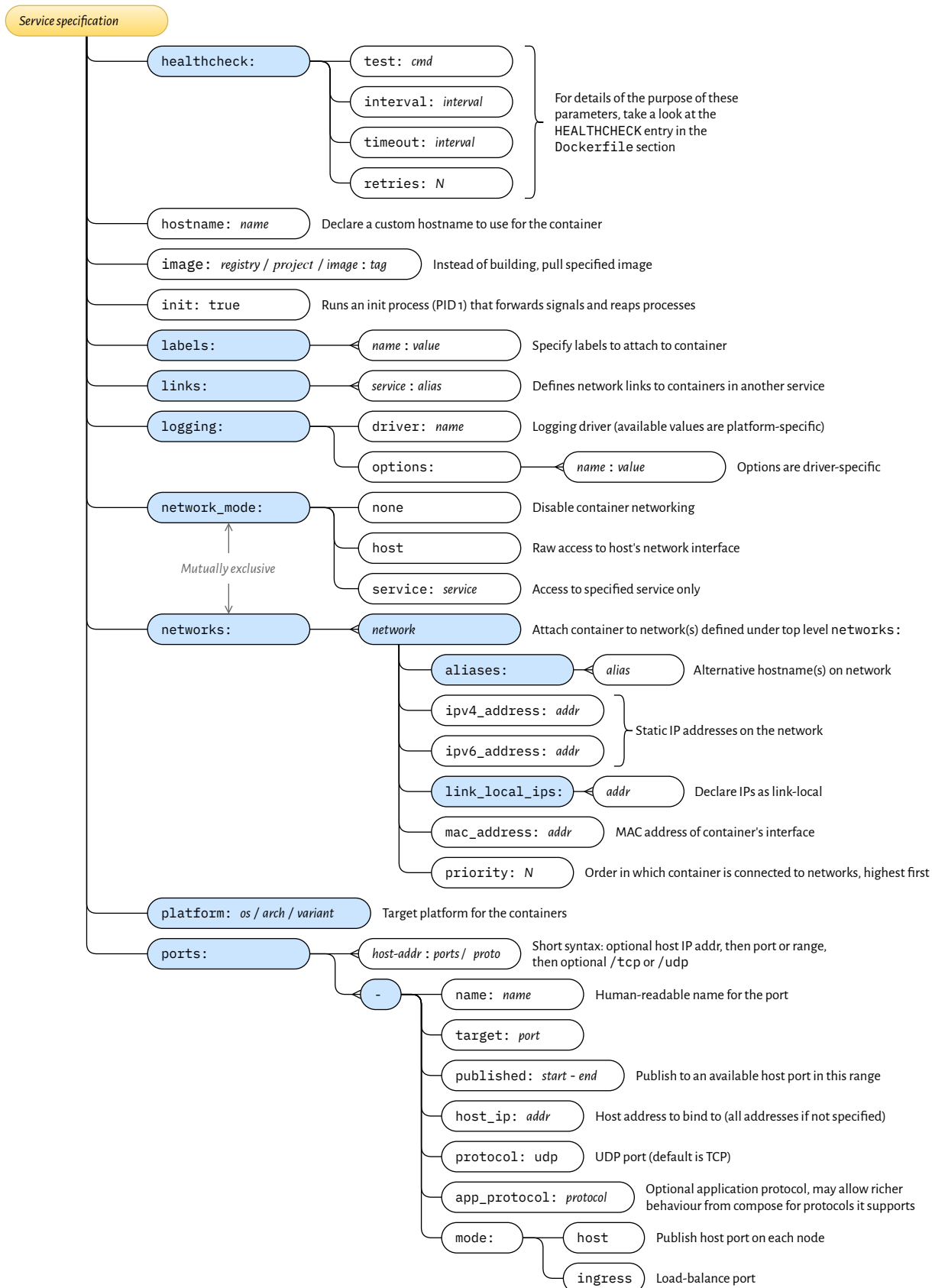




Last modified: 2024-04-13

© Copyright 2024 Andy Peace — <https://www.andy-pearce.com>Distributed under MIT license — <https://opensource.org/licenses/mit/>

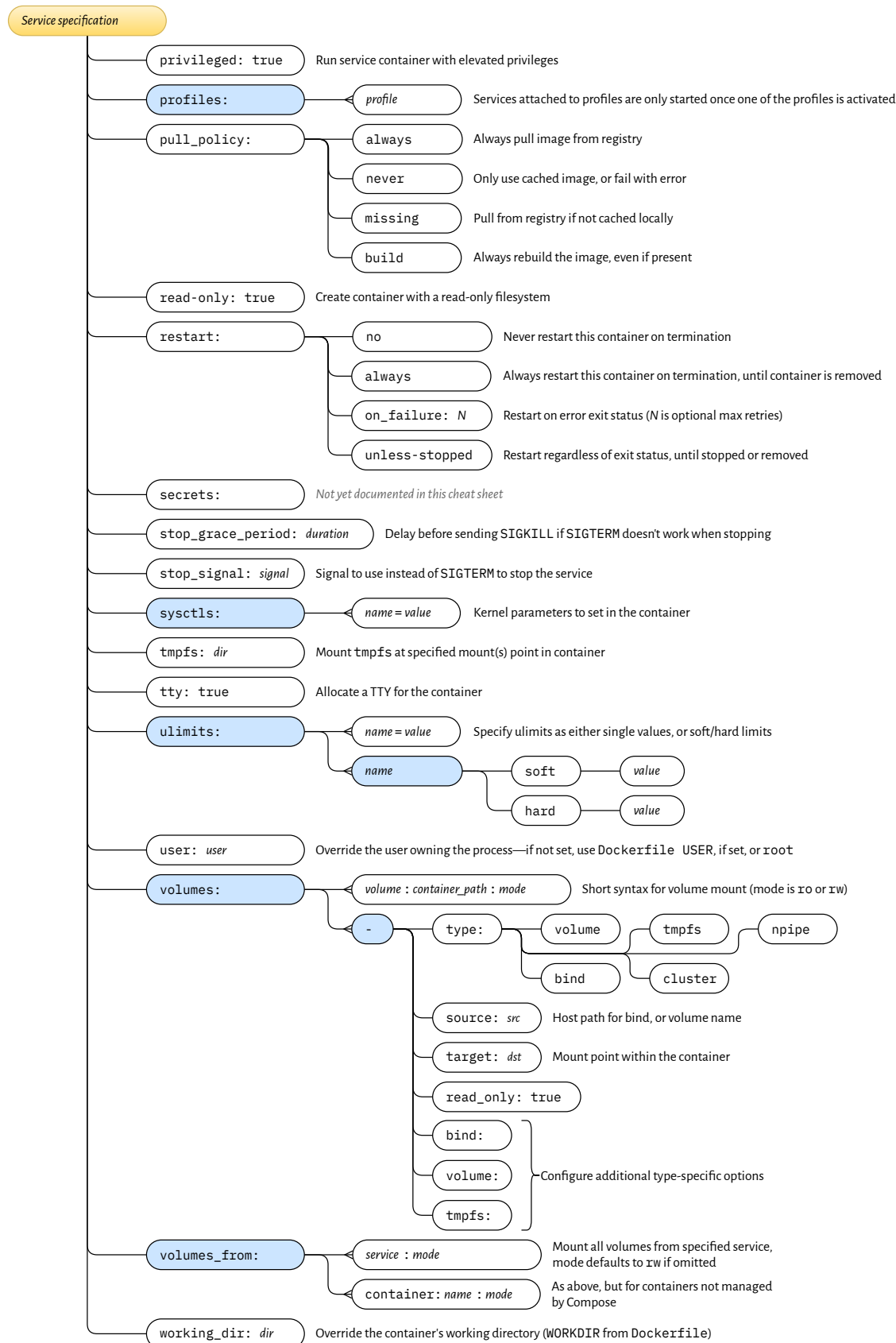
Docker Compose File (Service Specification, cont.)

Only some options are shown, this is not a comprehensive reference. Latest version: <https://www.andy-pearce.com/docker-cheat-sheet.pdf>

Docker and the Docker logo are trademarks or registered trademarks of Docker, Inc. in the USA and other countries.



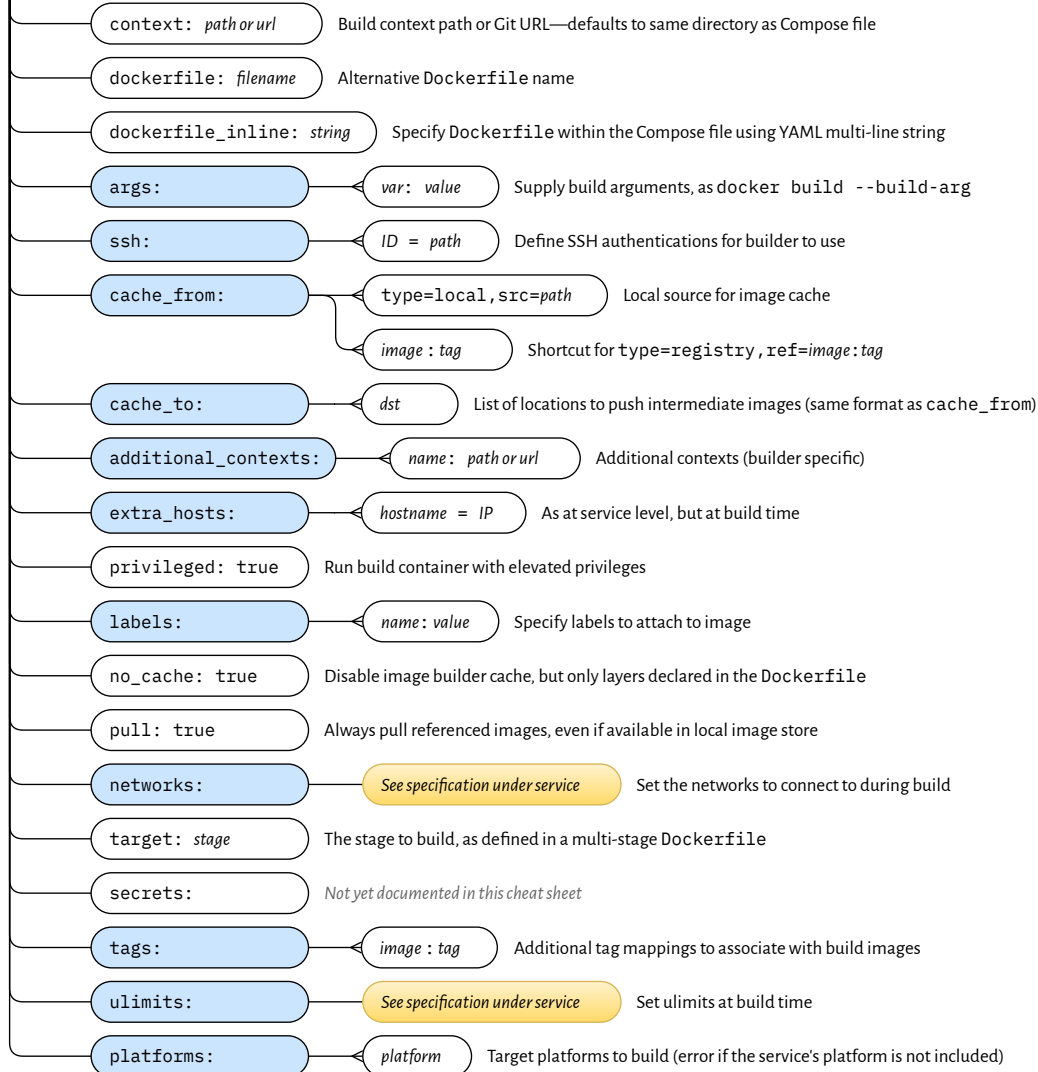
Docker Compose File (Service Specification, cont.)



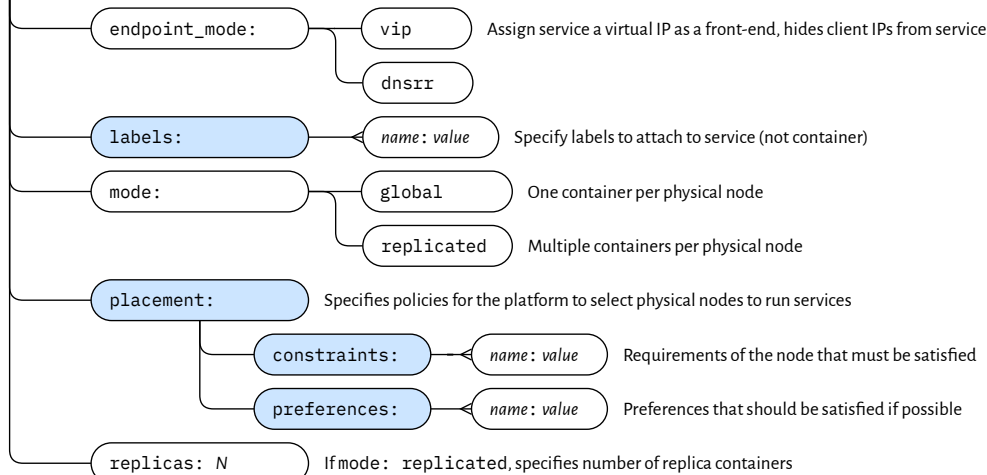


Docker Compose File (Service Specification, cont.)

Build specification

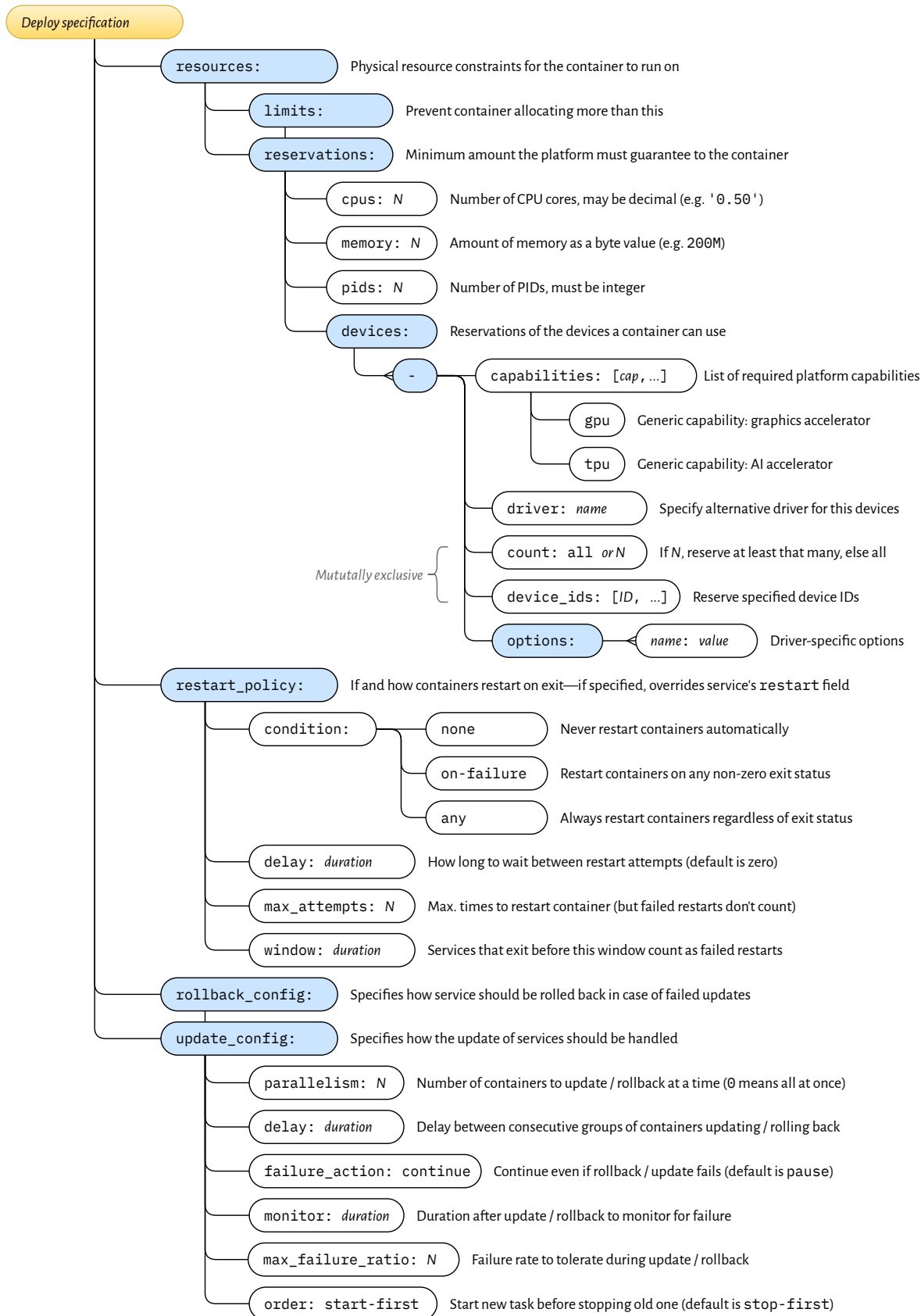


Deploy specification





Docker Compose File (Service Specification, cont.)





Docker Compose File (Network and Volume specifications)

